2600
2600, école de cybersécurité

MODULE
Noyau Windows

TP 04

Mini-rootkit guidé

Techniques offensives en mode kernel

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h

Sommaire

Objectifs . . . 3
Prérequis . . . 3
Rendu attendu . . . 4
Exercice 1 : DKOM — Masquer un processus (2h) . . . 4
Contexte . . . 4
Objectifs . . . 4
Ce que vous devez implémenter . . . 5
Header partagé (`dkom.h`) . . . 5
Driver (`dkom.c`) . . . 5
Programme userland (`hide.c`) . . . 5
Vérification étape par étape . . . 6
Exercice 2 : Token Stealing — Élévation de privilèges (1h30) . . . 6
Contexte . . . 6
Objectifs . . . 6
Ce que vous devez implémenter . . . 7
Driver (`steal.c`) . . . 7
Programme userland (`elevate.c`) . . . 7
Vérification étape par étape . . . 7
Exercice 3 : File Hiding via MiniFilter (1h30) . . . 8
Contexte . . . 8
Objectifs . . . 8
Ce que vous devez implémenter . . . 8
Logique de filtrage . . . 8
Squelette . . . 9
Fichier registre . . . 9
Vérification étape par étape . . . 9
Exercice 4 (bonus) : Rootkit unifié (30min) . . . 10
Barème indicatif . . . 10
Troubleshooting . . . 11

Objectifs

L'objectif de cet atelier est d'implémenter des techniques de rootkit en mode kernel Windows. C'est le TP préparatoire au projet Mini-Rootkit.

- DKOM : masquer un processus de la liste des processus
- Token stealing : élévation de privilèges vers SYSTEM
- File hiding : masquer des fichiers via minifilter

⚠ **Ces techniques sont à des fins éducatives uniquement.** Elles doivent être réalisées exclusivement sur votre VM de test.



Fig. 1. – width:400px

Prérequis

- TP 01 à 03 complétés
- VM Windows en mode testsigning/debug avec WinDBG connecté
- Maîtrise de : compilation de drivers, DeviceIoControl, minifilters
- Offsets `_EPROCESS` notés : `ActiveProcessLinks`, `Token`, `ImageFileName`

Rendu attendu

À la fin de ce TP, vous devez rendre une archive `tp04_NOM_PRENOM.zip` contenant :

Fichier	Description
`rapport_tp04.md`	Réponses aux questions, captures d'écran, analyse
`dkom/dkom.c`	Driver DKOM
`dkom/dkom.h`	Header partagé (IOCTL)
`dkom/hide.c`	Programme userland pour masquer un processus
`steal/steal.c`	Driver token stealing
`steal/steal.h`	Header partagé (IOCTL)
`steal/elevate.c`	Programme userland pour élever un processus
`hidefiles/hidefiles.c`	Minifilter de masquage de fichiers
`hidefiles/hidefiles.reg`	Fichier registre du minifilter

Le rapport doit contenir les réponses à toutes les questions numérotées, des captures d'écran **avant/après** chaque technique, et une analyse des limites.

Exercice 1 : DKOM — Masquer un processus (2h)

Contexte

`DKOM` (Direct Kernel Object Manipulation) consiste à manipuler directement les structures du noyau pour cacher des informations aux outils userland. Vous allez retirer un processus de la liste `ActiveProcessLinks` pour le rendre invisible au Task Manager et à `tasklist`.

Objectifs

Écrire un driver `dkom.sys` et un programme `hide.exe` qui :

1. `hide.exe <pid>` envoie le PID au driver via `DeviceIoControl`
2. Le driver trouve le `_EPROCESS` correspondant via `PsLookupProcessByProcessId`
3. Le driver retire ce processus de la liste doublement chaînée `ActiveProcessLinks`
4. Le processus continue de s'exécuter mais n'est plus visible dans le Task Manager

Ce que vous devez implémenter

Header partagé (`dkom.h`)

Définir un `IOCTL_HIDE_PROCESS` avec `CTL_CODE`.

Driver (`dkom.c`)

Squelette à compléter :

```c
#include <ntddk.h>
#include "dkom.h"

// TODO: Définir l'offset de ActiveProcessLinks
// Vérifiez avec WinDBG : dt nt!_EPROCESS ActiveProcessLinks
#define OFFSET_ACTIVE_PROCESS_LINKS   ???

void HideProcess(PEPROCESS Process)
{
    // TODO: Obtenir le pointeur vers ActiveProcessLinks du processus
    // PLIST_ENTRY current = (PLIST_ENTRY)
    //     ((ULONG_PTR)Process + OFFSET_ACTIVE_PROCESS_LINKS);

    // TODO: Sauvegarder les pointeurs prev (Blink) et next (Flink)

    // TODO: Faire pointer prev->Flink vers next

    // TODO: Faire pointer next->Blink vers prev

    // TODO: Faire boucler current sur lui-même (évite BSOD au cleanup)
    // current->Flink = current; current->Blink = current;
}

NTSTATUS DispatchIoControl(PDEVICE_OBJECT DeviceObject, PIRP Irp)
{
    // TODO: Récupérer le IO_STACK_LOCATION
    // TODO: Vérifier que l'IOCTL est IOCTL_HIDE_PROCESS
    // TODO: Lire le PID depuis Irp->AssociatedIrp.SystemBuffer
    // TODO: Appeler PsLookupProcessByProcessId pour obtenir le PEPROCESS
    // TODO: Appeler HideProcess
    // TODO: Ne pas oublier ObDereferenceObject !
    // TODO: Compléter l'IRP et retourner

    return STATUS_SUCCESS;
}

// TODO: Implémenter DriverEntry (device + symbolic link + MajorFunctions)
// TODO: Implémenter DriverUnload (cleanup)
```

Programme userland (`hide.c`)

Écrire un programme qui :

- Prend un PID en argument (`argv[1]`)
- Ouvre le device via `CreateFileW`

- Envoie le PID au driver via `DeviceIoControl`
- Affiche un message de confirmation

Vérification étape par étape

1. Lancer `notepad.exe` et noter son PID
2. Vérifier sa présence : `tasklist | findstr notepad`
3. Vérifier dans WinDBG : `!process 0 0 notepad.exe` (noter l'adresse EPROCESS)
4. Exécuter `hide.exe <pid>`
5. Vérifier la disparition : `tasklist | findstr notepad` → aucun résultat
6. Vérifier dans WinDBG : `!process 0 0 notepad.exe` → aucun résultat
7. Mais le processus tourne toujours : `!process <adresse_eprocess> 0` → toujours actif

Q1 : Faites des captures d'écran **avant** et **après** le masquage (Task Manager + sortie WinDBG). Incluez-les dans votre rapport.

Q2 : Le processus masqué est-il toujours visible via d'autres mécanismes ? Testez les commandes WinDBG suivantes et expliquez les résultats :

- `!process 0 0` (utilise ActiveProcessLinks → affecté)
- `dx @$cursession.Processes` (utilise le debugger data model → ?)
- `!object \Device` (namespace → ?)

Q3 : Quels sont les risques de cette technique ? Que se passe-t-il si le processus masqué se termine ? Y a-t-il un risque de BSOD ? Dans quelles circonstances ?

Exercice 2 : Token Stealing — Élévation de privilèges (1h30)

Contexte

Le champ `Token` de `_EPROCESS` contient un pointeur (`_EX_FAST_REF`) vers le jeton d'accès du processus. En copiant le token du processus **System** (PID 4) vers un processus utilisateur, ce dernier obtient les privilèges **NT AUTHORITY*.

Objectifs

Écrire un driver `steal.sys` et un programme `elevate.exe` qui :

1. `elevate.exe <pid>` envoie le PID cible au driver
2. Le driver copie le token du processus System (PID 4) vers le processus cible
3. Le processus cible obtient les privilèges SYSTEM

Ce que vous devez implémenter

Driver (`steal.c`)

Squelette à compléter :

```c
#include <ntddk.h>
#include "steal.h"

// TODO: Définir l'offset du champ Token dans _EPROCESS
// Vérifiez avec WinDBG : dt nt!_EPROCESS Token
#define OFFSET_TOKEN   ???

void StealToken(ULONG targetPid)
{
    PEPROCESS targetProcess = NULL;
    PEPROCESS systemProcess = NULL;

    // TODO: Trouver le processus System (PID 4) avec
PsLookupProcessByProcessId

    // TODO: Trouver le processus cible avec PsLookupProcessByProcessId

    // TODO: Lire le token de System à l'offset OFFSET_TOKEN
    // ULONG_PTR systemToken = ???

    // TODO: Écrire ce token dans le processus cible au même offset

    // TODO: ObDereferenceObject sur les deux processus
}

// TODO: Implémenter la dispatch function pour recevoir le PID via IOCTL
// TODO: Implémenter DriverEntry et DriverUnload
```

Programme userland (`elevate.c`)

Écrire un programme qui :

- Prend un PID en argument
- Envoie le PID au driver via `DeviceIoControl`
- Affiche un message de confirmation

Vérification étape par étape

1. Ouvrir un `cmd.exe` non-admin et noter son PID
2. Dans ce cmd : `whoami` → doit afficher votre utilisateur
3. Exécuter `elevate.exe <pid_cmd>` (depuis un autre terminal admin)
4. Dans le cmd ciblé : `whoami` → doit afficher `nt authority\system`

Q4 : Faites des captures d'écran de `whoami` **avant** et **après** le token stealing. Incluez-les dans votre rapport.

Q5 : Le type `_EX_FAST_REF` utilise les 4 bits de poids faible comme compteur de référence. Expliquez :

- Pourquoi ces bits ne font pas partie de l'adresse réelle du token ?
- Faut-il les masquer lors de la copie ? Que se passe-t-il si on copie la valeur brute (avec les bits de référence) ?
- Testez les deux approches et reportez les résultats.

Q6 : Après le token stealing, le processus a les privilèges SYSTEM mais l'utilisateur n'a pas changé dans la session Windows. Expliquez pourquoi `whoami` retourne « system » alors que la session de bureau reste celle de l'utilisateur original.

Exercice 3 : File Hiding via MiniFilter (1h30)

Contexte

Un minifilter peut intercepter les requêtes de listage de répertoire (`IRP_MJ_DIRECTORY_CONTROL`) et filtrer les résultats pour cacher des fichiers. Les fichiers existent toujours sur le disque mais ne sont plus visibles via `dir` ou l'Explorateur Windows.

Objectifs

Écrire un minifilter `hidefiles.sys` qui :

1. Intercepte `IRP_MJ_DIRECTORY_CONTROL` en **post-opération**
2. Filtre sur `IRP_MN_QUERY_DIRECTORY`
3. Parcourt le buffer `FILE_BOTH_DIR_INFORMATION` et retire les entrées contenant un mot-clé (ex: `"rootkit"`)
4. Les fichiers cachés restent accessibles par chemin direct

Ce que vous devez implémenter

Logique de filtrage

Le buffer retourné par le système est une **liste chaînée** de structures `FILE_BOTH_DIR_INFORMATION`. Chaque entrée a :

- `NextEntryOffset` : distance vers l'entrée suivante (0 si dernière)
- `FileName` / `FileNameLength` : nom du fichier (WCHAR)

Pour « supprimer » une entrée de la liste :

- Si c'est une entrée **intermédiaire** : ajouter son `NextEntryOffset` à celui de l'entrée précédente (on « saute » par dessus)

- Si c'est la **dernière** entrée : mettre le `NextEntryOffset` de la précédente à 0
- Si c'est la **première** (et seule) entrée : mettre `Data->IoStatus.Information` à 0

Squelette

```c
#include <fltKernel.h>

#define HIDE_KEYWORD L"rootkit"

// TODO: Implémenter la fonction ShouldHide(PWCHAR fileName, ULONG length)
// qui retourne TRUE si le nom contient HIDE_KEYWORD

// TODO: Implémenter PostDirectoryControl
// 1. Vérifier que MinorFunction == IRP_MN_QUERY_DIRECTORY
// 2. Vérifier que l'opération a réussi (NT_SUCCESS)
// 3. Récupérer le buffer depuis Parameters.DirectoryControl
//        .QueryDirectory.DirectoryBuffer
// 4. Parcourir la liste chaînée FILE_BOTH_DIR_INFORMATION
// 5. Pour chaque entrée à cacher, ajuster les pointeurs NextEntryOffset

// TODO: Déclarer le tableau FLT_OPERATION_REGISTRATION
//   → intercepter IRP_MJ_DIRECTORY_CONTROL en post-opération

// TODO: Déclarer la structure FLT_REGISTRATION

// TODO: Implémenter DriverEntry (FltRegisterFilter + FltStartFiltering)
// TODO: Implémenter FilterUnload (FltUnregisterFilter)
```

Fichier registre

Créer `hidefiles.reg` avec altitude `370000` (comme dans le TP 02).

Vérification étape par étape

1. Créer les fichiers suivants :
   - `C:\rootkit_payload.exe` (fichier vide)
   - `C:\rootkit_config.txt` (fichier vide)
   - `C:\legitimate_file.txt` (fichier vide)
2. `dir C:\` → les 3 fichiers sont visibles
3. Charger le minifilter
4. `dir C:\` → seul `legitimate_file.txt` est visible
5. `type C:\rootkit_payload.exe` → le fichier est toujours accessible directement
6. Décharger le minifilter
7. `dir C:\` → les 3 fichiers sont de nouveau visibles

Q7 : Faites des captures d'écran de `dir C:\` **avant** et **après** le chargement du minifilter.

Q8 : Testez les limites de votre technique :

- Le fichier est-il visible dans l'Explorateur Windows graphique ?
- Le fichier est-il visible via PowerShell (`Get-ChildItem C:\`) ?

- Le fichier est-il accessible via son chemin direct (`type C:\rootkit_payload.exe`) ?
- Un outil forensique comme `NtQueryDirectoryFile` direct (sans passer par l'API Win32) peut-il voir le fichier ?

Reportez les résultats dans votre rapport.

Q9 : Comparez cette technique de masquage de fichiers avec le masquage de processus (DKOM). Quelles sont les similitudes et les différences en termes de :

- Mécanisme utilisé (manipulation de structure vs filtrage d'IRP)
- Détectabilité
- Stabilité (risque de BSOD)

Exercice 4 (bonus) : Rootkit unifié (30min)

Combinez les 3 techniques dans un **driver unique** `rootkit.sys` avec un programme de contrôle `rootctl.exe` :

```
code

rootctl.exe hide-process <pid>
rootctl.exe steal-token <pid>
rootctl.exe hide-file <keyword>
rootctl.exe status
```

Définir 3 IOCTLs distincts dans un header partagé.

⚠ Le file hiding nécessite un minifilter séparé ou combiné dans le même driver (complexe). Vous pouvez le laisser comme driver séparé chargé par `rootctl.exe` via un script.

Q10 (bonus) : Ce driver unifié constitue le squelette de votre projet Mini-Rootkit. Identifiez 3 fonctionnalités supplémentaires que vous pourriez ajouter pour le projet, avec une description technique de leur implémentation.

Barème indicatif

Élément	Points
Exercice 1 — DKOM fonctionnel (Q1–Q3)	7
Exercice 2 — Token Stealing fonctionnel (Q4–Q6)	7
Exercice 3 — File Hiding fonctionnel (Q7–Q9)	6
Bonus — Rootkit unifié (Q10)	3
Total	**23** (ramené sur 20)

Troubleshooting

Problème	Solution
BSOD après DKOM	Vérifiez que vous faites boucler le processus sur lui-même (`Flink = Blink = current`)
L'offset ne correspond pas	Les offsets changent entre versions de Windows — toujours vérifier avec `dt nt!_EPROCESS` dans WinDBG
`whoami` ne change pas après token stealing	Vérifier que vous écrivez bien au bon offset. Utiliser WinDBG : `dt nt!_EPROCESS Token <addr>` avant et après
Le minifilter ne cache rien	Vérifier que vous interceptez bien `IRP_MJ_DIRECTORY_CONTROL` en **post**-opération (pas pre)
Le minifilter cache tout	Votre fonction `ShouldHide` a un bug — vérifier la comparaison de chaînes WCHAR (attention à la longueur)
`fltmc load` échoue avec « instance not found »	Le `.reg` n'a pas été correctement importé — vérifier les clés `Instances` et `DefaultInstance`